Medians and order statistics

Part II Sorting and Order Statistics

Scope and prerequisites

Selection finds one rank without sorting everything. Begin with exact extreme-value comparison counts, unify three-way regions and local ranks, prove expected linear random selection, and derive worst-case linear time from actual five-element exclusion witnesses. Prerequisites are selection, partitioning, and probabilistic analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Find both extremes with ceil(3n/2)−2 comparisons and prove a matching worst-case lower bound.
- Prove expected linear randomized selection.
- Use grouped medians for a worst-case linear guarantee.

Sections 9.1–9.3 cover extremes, expected linear selection, and worst-case linear selection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

9.1 Minimum and maximum

The k-th smallest value is position k of the sorted multiset, with $1 \le k \le n$. Duplicate occurrences occupy separate positions but may return the same value. The lower median for even n uses k=ceil(n/2), not necessarily the average of the central values. Keys must form a total order. Reject empty inputs and invalid k at entry.

A minimum scan maintains the smallest value of the read prefix and uses $n-1$ comparisons. For a lower bound, every identity except the actual minimum must be shown greater than something. One comparison can first eliminate at most one minimum candidate, requiring at least $n-1$ comparisons. Maximum is symmetric.

Both extremes need not cost $2n-2$. Compare pairs first; only the smaller challenges min and only the larger challenges max. For even n initialize from the first pair with one comparison. For odd n initialize both from the first record.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete paired comparison count

For [14,3,9,1,8,5,12,6], the first comparison gives min=3,max=14. Each further pair uses one internal, one min, and one max comparison.

Pair	Smaller, larger	New min	New max	Total comparisons
14,3	3,14	3	14	1
9,1	1,9	1	14	4
8,5	5,8	1	14	7
12,6	6,12	1	14	10

Even n gives 1+3(n−2)/2=3n/2−2; odd n gives 3(n−1)/2. Both equal ceil(3n/2)−2, including zero at n=1. After each pair the extrema cover a longer prefix, proving initialization, preservation, and exit just as for a scan, while sharing comparison information between the two tasks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matching worst-case lower bound

For distinct keys, use an adversary whose answers remain compatible with a total order. Each identity initially has both min and max candidacy. At exit only the true min's min candidacy and true max's max candidacy remain, so $2n-2$ candidacies must disappear.

Comparing two never-compared keys can eliminate two: the larger loses min and the smaller loses max. At most $\lfloor n/2 \rfloor$ such comparisons exist. In every other comparison the adversary can eliminate at most one new candidacy.

Before comparison	Answer rule
Both max-only candidates	Choose a greater one; eliminate the other's max status
Both min-only candidates	Choose a smaller one; eliminate the other's min status
One max-only, one min-only	Declare max-only greater; eliminate neither
One never compared	Eliminate one of its statuses without another new loss

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Matching worst-case lower bound (continued)

Keys with neither status cannot cause double elimination. The rules preserve the comparison partial order: remaining max candidates have never lost; min candidates have never won. Placing the former above the latter creates no directed cycle. Uncompared keys have no previous constraints.

With c comparisons, at most c+floor(n/2) statuses disappear. Thus $c \geq 2n-2-\text{floor}(n/2)=\text{ceil}(3n/2)-2$. The paired method attains this bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A tournament retains second-smallest candidates

Organize minimum comparisons as a balanced knockout tournament and save each winner's direct opponents. The second smallest must have lost directly to the minimum. Anyone else lost to another nonminimum key that is smaller, precluding second place.

For the eight-key example, first-round winners are 3,1,5,6; second-round winners 1,5; final winner 1. Its direct opponents are 9,3,5, whose minimum is 3. Seven tournament comparisons plus two list comparisons total nine.

For general $n \geq 2$, balanced scheduling with byes, never comparisons to fictitious values, uses $n-1$ comparisons. The champion has $d \leq \text{ceil}(\log_2 n)$ actual opponents; another $d-1$ comparisons suffice. Worst total is at most $n+\text{ceil}(\log_2 n)-2$. Assume distinct keys, or use (key,identity) to define the second identity under ties.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

9.2 Selection in expected linear time

Select recurses on only the answer side; Quicksort uses both. To handle duplicates, use three-way partition throughout. Array interval p..r is **0-based**, but k is always the **1-based rank within the current interval**.

```
partition3(A,p,r,x):
    lt=p; i=p; gt=r
    while i<=gt:
        if A[i]<x: swap(A[lt],A[i]); lt++; i++
        elif A[i]>x: swap(A[i],A[gt]); gt--
        else: i++
    return lt,gt
```

Before a test, A[p..lt−1]<x,A[lt..i−1]=x,A[i..gt] is unknown,A[gt+1..r]>x. A small-key swap expands the smaller and equal regions. A greater-key swap brings an unknown value from gt, so i must not advance. Equality only expands the equal region. Unknown length gt−i+1 decreases each iteration, proving termination. Swaps preserve the multiset, giving complete regions at exit, without internal sorting.

Lin Cing
github.com/xinhi6716/computer-science-self-study

Subtract two regions only when going right

```
select-random(A,p,r,k):
    assert 1<=k<=r-p+1
    while p<r:
        x=A[uniform-integer(p,r)]
        lt,gt=partition3(A,p,r,x)
        L=lt-p; E=gt-lt+1
        if k<=L: r=lt-1
        elif k<=L+E: return x
        else: k-=L+E; p=gt+1
    return A[p]
```

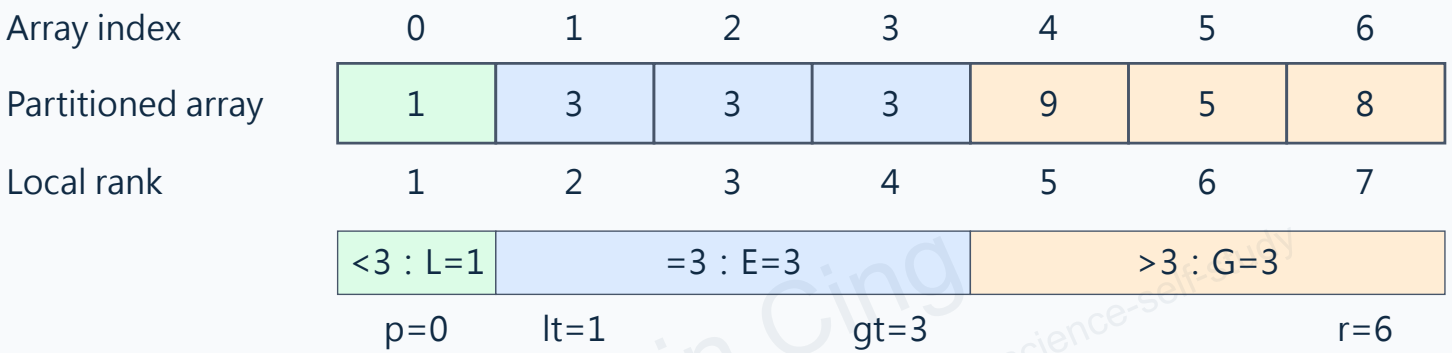
The pivot is a current record, so $E \geq 1$. A left branch has $k \leq L$; a right branch has $1 \leq k-L-E \leq r-gt$. Neither can select an empty interval, and size drops by at least E. Sorted local ranks L+1..L+E all equal x.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subtract two regions only when going right (continued)

Three-way partition: distinguish boundaries from local ranks

Input [8,3,5,3,9,1,3], pivot 3. Indices start at 0; local ranks at 1.



k=1: rank 1 left. k=2..4: return 3. k=6: rank 6−L−E=2 right.

Green <, blue =, orange >; interiors unsorted. For nonzero p, L=lt−p; lt is not a rank.

For A=[8,3,5,3,9,1,3],x=3, lengths are L=1,E=3,G=3. Rank 4 returns 3; rank 1 stays rank 1 left; rank 6 becomes rank 6−1−3=2 right. Absolute positions and local ranks differ. Another input [3,3,1,3,2,3] can become [1,2,3,3,3,3], with inclusive equal interval (lt,gt)=(2,5).

A selection that continues right

For A=[7,2,9,4,5],k=3, choosing x=2 gives empty smaller region, equal [2], and greater records [7,9,4,5]. With L=0,E=1, continue right looking for local rank 2, not 3.

If the next pivot is 5, the retained region divides into [4],[5],[7,9]. Its L=1,E=1 put k=2 in the equal region, returning 5. Discarded region interiors need not be sorted.

All-equal [5,5,5,5,5] ends after one $\Theta(n)$ scan. Two-way $\leq$ Lomuto selection for the minimum instead repeatedly has q=r, making 4+3+2+1=10 pivot comparisons and $\Theta(n^2)$ generally. Random choice among identical 5s cannot change that degeneration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete expected-linear substitution proof

Conditional on the current array, each pivot identity is uniform; future selections remain conditionally uniform. For distinct keys the rank R is uniform in 1..n. The retained length is at most m_R=max(R−1,n−R).

Let F(A,k) be expected remaining time. Assume every shorter state obeys F(B,j)≤a|B|, and current scanning plus fixed overhead costs at most cn. Condition on each pivot and apply induction:

$$F(A,k) \le cn + \frac{a}{n}\sum_{r=1}^{n} \max(r-1, n-r).$$

No monotonicity of an unknown T or independence of dependent costs is assumed. For n=2h, average m is $(3h^2-h)/(2h)=3n/4-1/2$. For n=2h+1 it is $(3h^2+2h)/(2h+1)\le 3n/4$. Thus F≤(c+3a/4)n≤an when a≥4c, also enlarged to cover n=1. The first scan gives Ω(n), proving expected Θ(n).

Replacing this argument by E[T(n)]≤E[T(3n/4)]+cn would be unjustified: an individual pivot can be extreme. The proof averages all conditional continuation bounds.

Duplicates and the worst-case boundary

The same proof handles duplicates by assigning equal keys a fixed identity order **only for analysis**, defining a unique pivot rank R. Actual partition still compares keys alone. The <x region has length at most R−1; the >x region at most n−R. Landing in =x ends immediately. Every pivot identity therefore has the same m_R bound and the same average induction; identities need not actually be sorted.

Worst execution remains $\Theta(n^2)$: with distinct keys, seek the minimum and happen to choose the maximum every time. Sizes n,n−1,… give the bound. A small but possible event cannot be excluded by expected linearity. Iterative selection avoids recursive frames; in-place partition plus fixed variables uses O(1) extra space.

Sorting first is a valid O(n log n) approach to medians or Top-k, but is not necessary. Returning one order statistic does not also produce a full sorting, so the sorting lower bound does not apply.

9.3 Selection in worst-case linear time

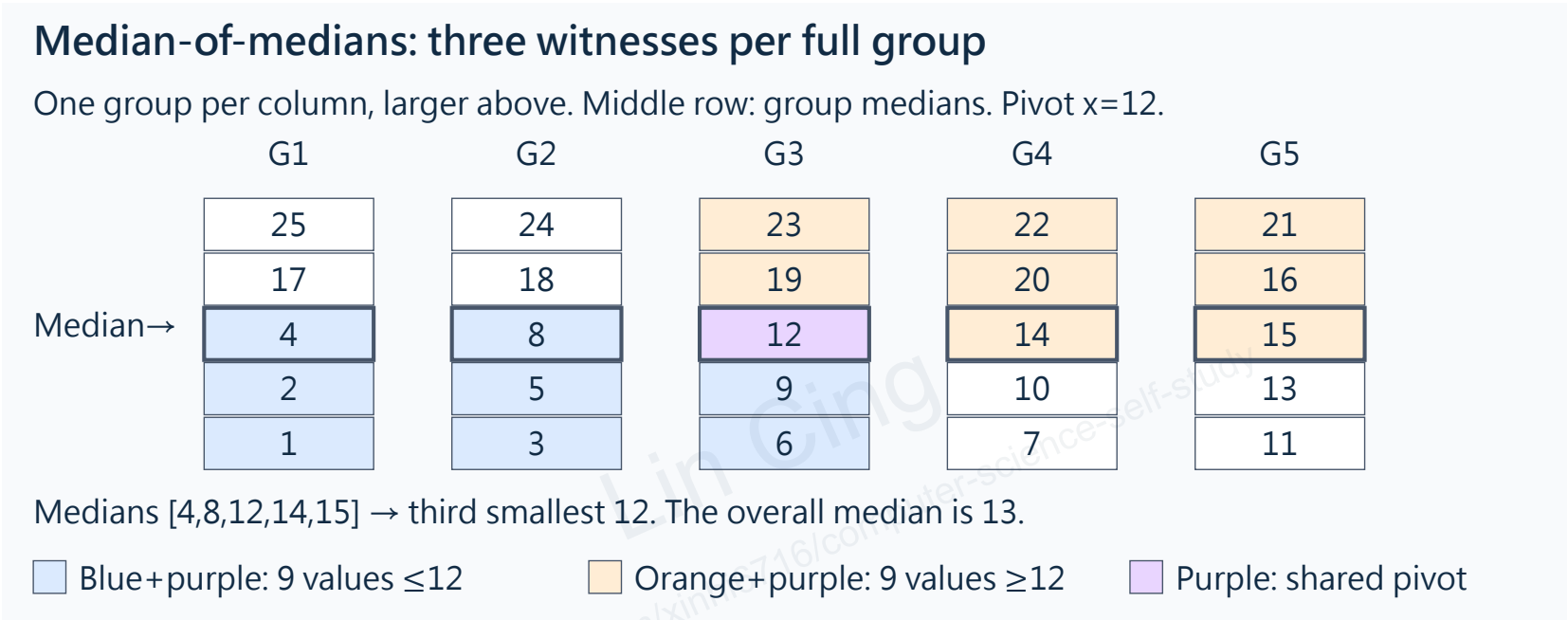
Deterministic select uses median-of-medians. For $n \leq 5$ sort directly and return rank k. Otherwise group current positions into sets of five, allowing one shorter final group. Sort each and take its lower median. Recursively select rank ceil(g/2) among these g=ceil(n/5) medians to choose x.

Use the same three-way partition. Return x if k is equal-region rank; otherwise recurse into the strict smaller or greater side with the same local-rank conversion. Both the pivot-selection recursion and answer recursion contribute to cost.

Each constant-size group sorts in O(1), totaling O(n). The pivot is an original value, so $E \geq 1$. Partition invariants and induction establish correctness and termination. The following exclusion count establishes efficiency separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Five-element exclusion witnesses



The five sorted groups have medians [4,8,12,14,15]. Their median is 12, whereas the entire input's true median is 13. The pivot needs a sufficiently balanced split, not an immediate answer.

Five-element exclusion witnesses (continued)

The first three groups have medians ≤ 12, each providing three values ≤ 12, giving nine witnesses. The last three have medians ≥ 12, each providing three values ≥ 12. The two witness sets may overlap at the pivot; they bound separate alternative recursive branches.

With duplicates the guarantees are nonstrict $\leq x$ and $\geq x$. A strict-side recursion excludes the full equal region, so these witnesses still bound its size.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rounding, short groups, and linear induction

At least ceil(g/2) group medians are ≥x and at least ceil(g/2) are ≤x, even with a lower median at even g. Conservatively remove the possible short group and the chosen pivot's group. At least ceil(g/2)−2 full groups remain, each supplying three nonstrict witnesses.

Each strict recursion therefore excludes at least 3(ceil(g/2)−2)≥3n/10−6 records, leaving at most 7n/10+6. A negative exclusion lower bound for small n is merely uninformative; it does not mean a growing subproblem.

With nonrecursive work ≤cn and induction T(m)≤am for smaller sizes, n≥140 gives

$$T(n) \le a\lceil n/5 \rceil + a(7n/10 + 6) + cn \le (9a/10 + c)n + 7a.$$

Choose a≥20c. Then a/10−c≥a/20, and n≥140 pays 7a, giving T≤an. Increase a to cover finitely many sizes below 140. This is a proof threshold, not a requirement to sort 140-record bases in the implementation. Both recursive sizes are smaller, making induction valid. Processing n records gives Ω(n), hence worst-case Θ(n).

Complete path to the sixteenth smallest

Use stable scan-order lists L/E/G for readability. In-place swaps may reorder within regions but preserve lengths and rank conversions.

Current n,k	Group medians → pivot	L,E,G sizes	Next step
25,16	[4,8,12,14,15]→12	11,1,13	rank 4 in greater region
13,4	[19,20,16]→19	6,1,6	rank 4 in smaller region
6,4	[15,16]→lower median 15	2,1,3	rank 1 in greater region
3,1	base sort [16,17,18]	—	return 16

The first greater list is [17,25,18,24,19,23,14,20,22,13,15,16,21]. Groups of lengths 5,5,3 give medians 19,20,16. The subsequent smaller list is [17,18,14,13,15,16]; its first five have median 15 and its singleton median 16. Pivot 15 leaves greater list [17,18,16]. The answer 16 agrees with position 16 of a full sorting used only as a check.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Different group sizes require new proofs

For fixed odd group size s=2t+1, an idealized large-n half of group medians supplies t+1 excluded records per group. The two recursive size fractions sum approximately to

$$1/s + 1 - (t+1)/(2s) = 1 + (1-t)/(2s).$$

For s=5 this is 9/10<1, leaving induction slack. For s=7 it is 6/7<1, also permitting linearity after recomputing constants and short groups. At s=3 it is 1. The coarse recurrence T(n)≤T(n/3)+T(2n/3+O(1))+O(n) proves O(n log n), but does not alone prove an actual Ω(n log n) runtime. A nonlinearity claim needs a matching family sustaining bad splits across recursive levels.

A straightforward list implementation stores groups, medians, and L/E/G with O(n) peak extra space, since active recursive path sizes shrink geometrically. In-place O(1) space would require additional median movement and stack design; it is not part of the stated time theorem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 9.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study